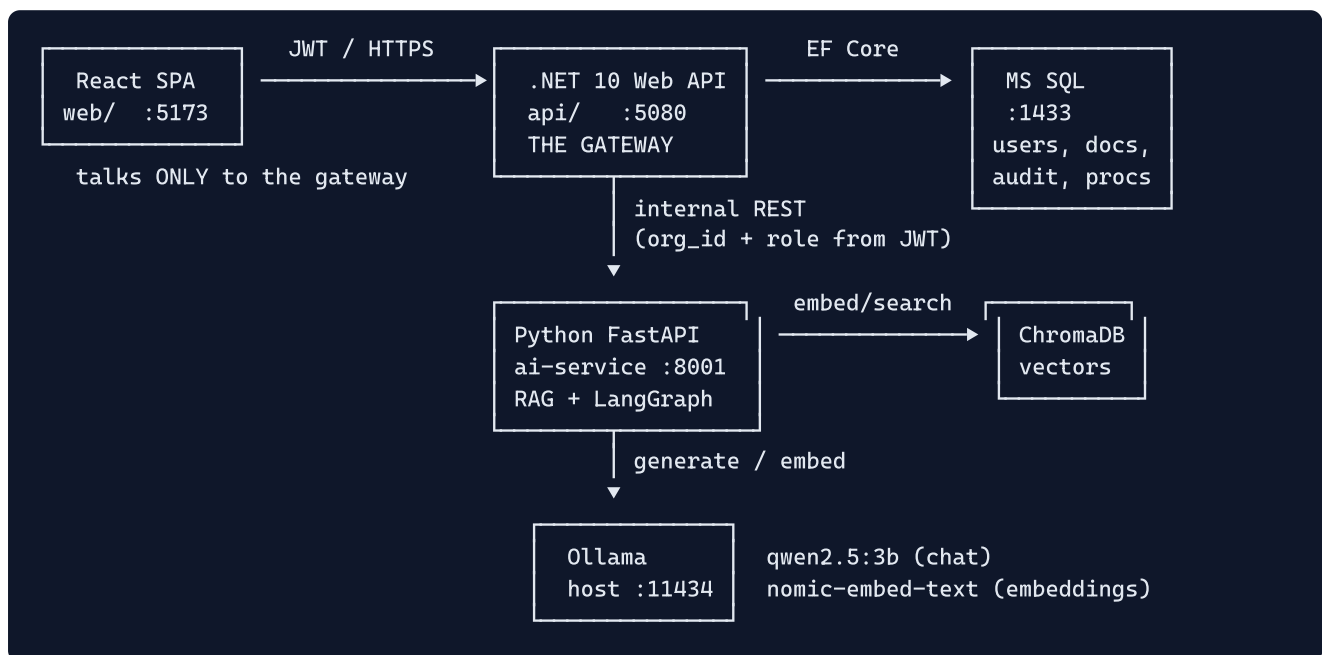


How It Works

A plain-language walkthrough of the Business Process Automation system, traced through the real code. For the formal reference see [ARCHITECTURE.md](#); to run it see [SETUP.md](#).

The big picture: 3 services + 2 datastores



The single most important design rule: **the browser only ever talks to the .NET gateway**. The Python AI service is never exposed to the client. Why this matters is the security model below.

The security boundary (the thesis's core claim)

The Python service is a **stateless "AI brain."** It does not decide who is allowed to see what — it just takes `{org_id, role_level, query}` and answers. The gateway is the one place that establishes identity:

1. On login, the .NET API issues a **JWT** carrying `sub` (user id), `org`, and `role` claims (HMAC-SHA256).
2. When the browser makes an AI call, it sends only the question. The gateway reads `org_id` and `role` from the **verified token**, never from the request body:

```
csharp // AiController.cs var result = await ai.RunAgentAsync(currentUser.OrgId,
currentUser.Role, req.Query, ct);
```

3. The AI service filters retrieval to `org_id == caller.org AND access_role ≤ caller.roleLevel`.

So a user **physically cannot widen their own access** — the role is bound to a signed token they cannot forge, and it is the gateway (not the client) that passes it down. An Employee asking for a Manager-only document gets zero matching chunks at the retrieval layer, so the model literally never sees that text. That is real RBAC, not a UI hide.

Role levels: `Employee = 0 < Manager = 1 < Admin = 2`.

What lives where

Store	Holds
MS SQL	Users (BCrypt hashes), Organizations, Document <i>metadata</i> (status, AccessRole), BusinessProcesses, AuditLog
Disk (<code>data/uploads/<orgId>/</code>)	The actual file bytes
ChromaDB	Embedded chunks + vectors, each tagged <code>{org_id, doc_id, file_name, access_role, chunk_index}</code>

Structured state is .NET's job; vectors are the AI service's job. They never share a database.

Flow 1 — Uploading a document (building the knowledge base)

```
React: pick file + choose access role
→ POST /api/documents (multipart, JWT)
  → DocumentService: save bytes to disk, write metadata row (Status=Processing)
    → AiServiceClient.IngestAsync → POST /ingest on the AI service
      → extract text (pypdf / python-docx / openpyxl / plain)
      → chunk (LangChain RecursiveCharacterTextSplitter, 800 chars / 120 overlap)
      → embed each chunk via Ollama nomic-embed-text (768-dim)
      → store in ChromaDB with {org_id, access_role, ...}
    ← chunk count
  → update row Status = Indexed (or Failed)
```

The `access_role` chosen at upload time is baked into every chunk's metadata — that is what makes role-filtering possible later.

Flow 2 — Asking the AI assistant (the agentic path)

```
React: type a request
→ POST /api/ai/agent (JWT attached)
  → gateway validates JWT, reads org_id + role
  → POST /agent/run {org_id, role_level, query}
    on the AI service

    LangGraph: router → conditional edge → agent

  ← {route, answer, used_context, sources, structured}
  ← relayed unchanged
  ← UI renders: route badge, grounded answer, sources, structured JSON
```

Inside the AI service, this is a compiled **LangGraph** (`graph.py`):

```
router
route=="policy_qa"    → policy_qa
route=="doc_summary" → doc_summary
route=="email_draft" → email_draft → END
route=="report_draft"→ report_draft
route=="invoice_gen" → invoice_gen
```

- **Router** — keyword heuristic with an LLM fallback; classifies the request into exactly one agent key.
- **policy_qa / doc_summary / email_draft / report_draft** — all go through RAG, so they only ever see role-scoped context.
- **invoice_gen** — produces structured JSON, but **totals are computed server-side** — the LLM only extracts line items, arithmetic is never trusted to the model. (The proposal flags financial operations as the highest hallucination risk.)

The anti-hallucination mechanism

The RAG agent's system prompt (`rag/service.py`):

```
"You are a business assistant. Answer ONLY using the provided context. "
"If the context does not contain the answer, say you don't have that information. "
"Be concise and cite facts from the context. Do not invent details."
```

And critically, if retrieval returns **nothing** (e.g. the answer is in a doc your role cannot see), the service short-circuits *before even calling the LLM*:

```
if not sources:
    return RagAnswer(answer="I don't have any information on that...",
                      sources=[], used_context=False)
```

So "I don't know" is a real outcome, not a hallucinated guess. Every answer also returns its `sources` (which chunks were used) so claims are traceable — that is the grounding the thesis argues for.

Why these choices

- **Local Ollama only** → no API cost, data stays on-premise (the project's hard constraint).
- **Ollama embeddings** (not sentence-transformers) → avoids a multi-GB PyTorch install on a modest GPU.
- **Embedded ChromaDB 1.x** → prebuilt Rust wheels, no C++ toolchain on Windows.
- **.NET as the only client-facing service** → one security boundary.
- **Single-org MVP but every row carries `OrgId`** → multi-tenant later with no rewrite.

Measured results (meets the thesis bar)

Metric	Target	Result
Answer accuracy	≥ 80%	100% (10/10)
Latency (p95)	< 3 s	~2.0 s (avg ~1.75 s)

Source: `ai-service/eval/run_eval.py` , local `qwen2.5:3b` + `nomic-embed-text` on a modest GPU.